

GROUP 6 — FITNESS TRACKER

Dockerized 3-Tier Application with Full CI/CD (<https://github.com/priscy-dev/group6-dockerized-3tier-app.git>)

Cloud & DevOps Bootcamp — Capstone Project

Project Documentation & Final Report

Prepared and compiled by Rejoice, on behalf of Group 6

1. Executive Summary

Group 6 built a Dockerized three-tier Fitness Tracker application consisting of a React/Vite frontend, a Node.js/Express backend, and a MongoDB database, satisfying the compulsory capstone brief for all groups.

The project was delivered in two stages. The team first stood up a working baseline: three containerized services, a Docker Hub registry, a single Linux VM, and a GitHub Actions pipeline deploying over SSH. Once that baseline was proven end-to-end, the team deliberately re-architected the deployment onto AWS, replacing the VM/SSH/Docker Hub model with a production-grade design using Terraform-managed infrastructure, Amazon ECR, keyless GitHub OIDC authentication, and AWS Systems Manager for deployment — removing SSH exposure entirely.

This document presents the final, currently deployed architecture as the system of record, and preserves the earlier baseline as a documented historical stage, showing the reasoning and evidence behind the team's design evolution — in keeping with the brief's emphasis on designed systems rather than systems that merely happen to work.

The application supports user registration, login/logout, profile management, and workout tracking, with MongoDB as the authoritative data store, and has been verified to persist data across browser cache clears and redeployments.

2. Repository Setup and Team Collaboration

Prepared by: Priscilla

2.1 Objective

Create and organize the GitHub repository for the Group 6 project, enable collaborative development across the team, and establish a reviewed pull-request workflow rather than direct commits to main.

2.2 Activities Carried Out

- Created the GitHub repository group6-dockerized-3tier-app and initialized it with a README.md.
- Created frontend/, backend/, database/, and .github/workflows/ folders, each seeded with a README.md.
- Collected GitHub usernames from all team members and added them as collaborators, with follow-up reminders sent to members who had not yet responded.
- Confirmed the repository structure was organized and ready for parallel development.

2.3 Branch Protection and Review Practice

main was configured as a protected branch requiring at least one approving review before merge. All contributions were made on feature branches and merged via reviewed pull requests. This was tested directly: an early direct push to main was rejected by GitHub with “At least 1 approving review is required by reviewers with write access,” confirming the protection was active. The commit was moved to a branch, a pull request opened, reviewed, and merged.

2.4 Example Review: Frontend Pull Request

Field	Detail
Title	Add fitness tracker React frontend
Contributor	Nowos10
Branch merged into	main
Review steps	Opened PR, reviewed description and Files changed tab, confirmed changes were scoped to frontend/, verified no merge conflicts, approved and merged.

This was the first successful reviewed pull request for the project and established the review pattern used for all subsequent contributions, later extended to CODEOWNERS-based review for infrastructure changes.

3. Phase 0 Design Worksheet

This worksheet documents the design decisions made before implementation, as required by the brief. It reflects the reasoning actually used to build the system described in this report.

3.1 Tier Boundaries

The application is split into three independent containers rather than one combined process:

Tier	What lives here	Why separate
Frontend	React/Vite UI: profile, workout entry, workout history, statistics, generated workout plan. Served in production by Nginx, which also reverse-proxies /user, /api, and /health to the backend.	Presentation logic changes far more often (UI tweaks, new views) than business logic. Keeping it separate means a frontend release never risks breaking the API or database, and it can be rebuilt/redeployed independently.
Backend	Node.js/Express API: registration, login/logout (JWT/session via passport), workout CRUD, request validation, rate limiting.	All business logic and authentication live in one place, so rules are enforced consistently no matter which client calls the API. It can scale or restart independently of the UI and never talks to the browser directly — only Nginx does.
Database	MongoDB: user accounts, profiles, workout records. Runs with persistent EBS-backed storage on a private host with no public route.	Data must outlive both application deployments and container restarts. Isolating it on a private subnet with no direct internet route means even a fully compromised frontend or backend container cannot expose the database directly — only the backend, using credentials from Secrets Manager, can reach it.

Because each tier is independently replaceable, a deployment can update the frontend and backend images without touching the database at all — which is also why database data survives every redeploy.

3.2 Versioning and Tagging Strategy

Every image is tagged with the immutable short Git commit SHA that produced it (e.g. 5bf8a56), never only "latest". Both the frontend and backend images built from the same release always share the same SHA tag, so one number identifies the exact code running across the whole application, not just one tier.

Knowing what's live: the deployed SHA is written to AWS Systems Manager Parameter Store at deploy time and exposed publicly through a /version endpoint, so anyone — including an evaluator — can confirm exactly what's running with a single request, without needing server access.

```
curl --fail https://<application-domain>/version
aws ssm get-parameter --name /fitness-tracker/dev/current-release --region ca-central-1 --query Parameter.Value --output text
```

Rollback procedure, step by step:

- Identify a previously healthy SHA (recorded in GitHub Deployments history and/or the Parameter Store value from before the bad release).
- Open GitHub Actions → Rollback workflow → Run workflow, and enter that SHA.
- The workflow verifies both the frontend and backend images for that SHA still exist in ECR before doing anything — it fails safely rather than deploying a missing image.
- The deploy step runs through SSM against the application instance, exactly like a normal release, but using the older, already-built images — no rebuild happens.

- Confirm recovery via /health and /version, which should now report the rolled-back SHA.

3.3 Secrets Handling Plan

Secret	Where it lives	Notes
MongoDB credentials	AWS Secrets Manager	Injected into the backend container at runtime; never written into a Dockerfile or compose file
JWT signing key pair	AWS Secrets Manager	Generated separately from application code; rotated independently of releases
AWS deployment credentials	Not stored anywhere	GitHub Actions authenticates to AWS via OIDC — short-lived, workload-scoped tokens issued per run, so there is no long-lived AWS key to leak
GitHub → AWS role trust	IAM role trust policy (Terraform-managed)	Scoped to this repository and branch/environment only
Terraform state	Encrypted S3 backend with locking	Kept out of source control; access controlled via IAM

No secret of any kind is hardcoded in a Dockerfile, committed to the repository, or printed in pipeline logs. This was a deliberate improvement over the project's first working version, which relied on static GitHub-secret-stored SSH keys and a Docker Hub token — still off-repo, but a weaker model than the short-lived OIDC credentials used in the final design (see Section 4).

4. Project Evolution: From Baseline to Production Architecture

This section is historical. It documents the first working version of the pipeline and explains why the team moved beyond it. The system described here was superseded by the architecture in Section 4, which is what is currently deployed.

4.1 Stage 1 — Docker Hub + Single VM + SSH (Baseline)

Prepared by: Godellena Sam El-Ebs and Rejoice Dunu (CI/CD implementation), with early containerization work by CreativeLit.

The first working pipeline used the following flow, triggered on push or pull request:

- Validation: checkout, install dependencies, lint, and production build of the frontend.
- Build & Push: build the frontend Docker image, tag it with both latest and the Git short SHA, and push to Docker Hub.
- Deploy: SSH into a single AWS EC2 instance, copy the Docker Compose file, pull the new image, restart the containers, and run a smoke test against the public IP.

Secrets (DOCKERHUB_USERNAME, DOCKERHUB_TOKEN, EC2_HOST, EC2_USER, EC2_SSH_KEY, EC2_SSH_PORT) were stored as encrypted GitHub repository secrets, configured by the group lead. This stage proved the core CI/CD concept requested by the brief: automatic build, versioned tagging, automated deploy, and a working rollback path.

4.1.1 Issues identified at this stage

Issue	Cause
Deployment step failures	Missing/misconfigured repository secrets, blocked SSH port 22 in the security group, Docker not yet installed on the EC2 host, or invalid Docker Hub credentials.
Duplicate pull request could not be created	GitHub appends new commits to an existing open PR between the same branches rather than creating a second one — expected behavior, not a defect.

4.2 Why the team moved to AWS/Terraform/OIDC

Reviewing the baseline against the brief's requirements (traceable versioning, reliable rollback, no hardcoded secrets, minimal attack surface) surfaced three specific limitations the team chose to address:

- Open SSH (port 22) on the production host was a standing attack-surface the brief's own compliance-style thinking discouraged, even though it worked.
- Long-lived SSH keys and Docker Hub tokens stored as static GitHub secrets are a weaker security model than short-lived, workload-scoped cloud credentials.
- Infrastructure created by hand (one VM, manual security group rules) was not reproducible or peer-reviewable the way Terraform-defined infrastructure is.

These findings led directly to the redesign documented in Section 4, which the team treated as a controlled infrastructure migration rather than a rebuild — carried out on a dedicated feat/aws-cicd-rebuild branch and merged only after review.

5. Final System Architecture (Currently Deployed)

5.1 Summary

The final deployment is a production-oriented AWS architecture with private application and database hosts, HTTPS public access, immutable container releases, and pull-request-controlled delivery, provisioned entirely through Terraform. GitHub Actions authenticates to AWS via OpenID Connect (OIDC) — no long-lived AWS access keys are stored in GitHub. Images are published to Amazon ECR and deployed to the private application instance through AWS Systems Manager (SSM), removing the need for SSH or a bastion host.

5.2 Architecture Layers

Layer	Implementation	Purpose
Edge & DNS	Route 53, ACM, HTTPS Application Load Balancer	TLS termination and public routing for the custom domain
Network	VPC with public, application, and database subnets across two Availability Zones	Network isolation and future resilience
Application	Private Amazon Linux EC2 instance running Docker Compose	Hosts frontend and backend containers
Database	Private Amazon Linux EC2 instance, MongoDB, persistent EBS storage	Stores accounts, profiles, workouts, and application data
Images	Separate private Amazon ECR repositories	Stores immutable frontend and backend images
Secrets	AWS Secrets Manager	Stores MongoDB and JWT secrets outside the repository
Deployment	GitHub Actions OIDC + AWS Systems Manager	Keyless AWS authentication and private-host deployment
State	S3 Terraform backend	Remote infrastructure state and team-safe operations

5.3 Bastion Decision

No bastion host is part of the final architecture. Both EC2 instances are SSM-managed and reachable through Session Manager, which reduces public attack surface, eliminates SSH-key distribution, and avoids the cost and maintenance of a jump host.

5.4 Runtime and Persistence Flow

- A browser connects to the custom HTTPS domain through Route 53 and the Application Load Balancer.
- The frontend calls the backend API through the same public application endpoint.
- The backend authenticates the user and reads/writes MongoDB on the private database host.
- MongoDB data is stored on persistent EBS-backed storage, independent of browser cache.
- A deployment replaces application containers with images identified by one Git commit SHA; database data remains intact across releases.

5.5 Final Acceptance Criteria

- Pull requests validate frontend, backend, Docker configuration, and Terraform formatting/validation before merge.
- Merges to main create immutable frontend and backend images tagged with the Git commit SHA.
- GitHub Actions authenticates to AWS without long-lived access keys, using OIDC.

- Releases deploy through SSM; no public SSH port or bastion host is required.
- The public HTTPS endpoint and /health and /version endpoints pass after deployment.
- Registered users, profiles, and workouts persist in MongoDB across browser cache clearing and redeployment.
- A previous immutable image tag can be selected for rollback without rebuilding it.

6. CI/CD Pipeline (Final Version)

The workflow set in `.github/workflows/` performs the complete application release, split across `ci.yml`, `build-images.yml`, `release.yml`, `deploy.yml`, and `rollback.yml`.

6.1 Pipeline Stages

Workflow	Trigger	Responsibility
CI	Pull request	Lint, test, build, Docker validation, and Terraform checks — no production deployment
Build images (reusable)	Called by release	Build frontend and backend images and publish immutable SHA tags to ECR
Deploy (reusable)	Called by release or rollback	Verify both tags, start an SSM command, wait, and verify health/version
Release	Merge/push to main	Orchestrate build and production deployment, then record the GitHub deployment
Rollback	Manual workflow_dispatch	Redeploy an earlier verified Git SHA without rebuilding images

6.2 Standard Change Path

```
git switch -c feat/<short-description>
git add <changed-files>
git commit -m "feat: describe the change"
git push -u origin feat/<short-description>
```

- Open a pull request with base main and the feature branch as compare.
- Review the Files changed tab and wait for required status checks.
- Request and receive the required approval.
- Merge the pull request — the release workflow builds and deploys the merge commit.
- Confirm the deployment job, application health, and reported version before declaring success.

6.3 Production Version and Rollback

The production version is the immutable Git SHA used for both image tags, recorded in AWS Systems Manager Parameter Store and exposed through the `/version` endpoint. Verified successful deployment evidence used release `5bf8a56`.

```
curl --fail https://<application-domain>/health
curl --fail https://<application-domain>/version
aws ssm get-parameter --name /fitness-tracker/dev/current-release --region ca-central-1 --query Parameter.Value --output text
```

To roll back: open GitHub Actions → select Rollback → Run workflow → enter a previously healthy Git SHA → confirm the production environment approval if required. The workflow verifies both ECR images exist before deploying them — a rollback that references a missing image fails safely instead of deploying a broken release.

7. Terraform and AWS Provisioning

Terraform was executed from `infrastructure/terraform/envs/dev`, with provider checksums made portable for Linux, remote state stored in S3, and saved plan files treated as local, single-use artifacts.

```
cd infrastructure/terraform/envs/dev
```

```
terraform fmt -check -recursive ../..  
terraform init -reconfigure  
terraform validate  
terraform plan -out=dev.tfplan  
terraform apply dev.tfplan  
terraform output
```

7.1 Recovery and Final Infrastructure Changes

- Recreated the database and application instances after the failed initial provisioning path.
- Increased root volume capacity to prevent Docker/package/startup exhaustion; the database also retains a separate persistent data volume.
- Attached the application instance to the load balancer target group and confirmed SSM registration.
- Created/updated the GitHub deployment IAM role, OIDC trust relationship, ECR permissions, and SSM deployment permissions.
- Exported instance IDs, repository names, application URL, current-release parameter, role ARN, region, and Terraform state bucket as Terraform outputs.

8. Backend Implementation and Testing

8.1 Stack and Structure

The backend was implemented with Node.js, Express, and MongoDB (via Mongoose), using a modular folder structure (controllers, routes, config, db, middleware, lib, utils, modules) for clear separation of concerns.

```
npm init -y
npm install express mongoose dotenv cors cookie-parser passport compression express-rate-limit
```

Package	Role
express	Server and routing
mongoose	MongoDB connection and modeling
dotenv	Environment variable management
cors	Cross-origin access control
cookie-parser	Cookie handling for auth
passport	Authentication
compression	Response performance
express-rate-limit	Basic abuse/rate protection

8.2 API Verification (Postman)

Test	Endpoint	Result
User registration	POST /user/register	Registration successful; user stored in MongoDB
User login	POST /user/login	Login successful; JWT issued and stored as an httpOnly cookie; authenticated state confirmed
Logout	—	JWT cleared; session invalidated
Workout submission	POST (authenticated)	Workout stored in MongoDB; route correctly required authentication

Example workout payload used for testing:

```
{
  "form": {
    "exercise": "Bench Press",
    "sets": 4,
    "reps": 8,
    "weight": 60
  }
}
```

8.3 Data Persistence Correction

Registration reached MongoDB from an early stage, but the initial frontend displayed profile and workout data from browser localStorage. This produced a misleading partial integration: users could register, yet clearing browser storage removed the displayed data even though the account still existed server-side.

Corrected behavior: authentication uses the backend session/JWT mechanism; login restores the authenticated user's profile and workouts from the API; all create/edit/delete actions go through authenticated endpoints; MongoDB is the single source of truth, with browser storage limited to non-authoritative UI/session hints.

Persistence verification test

- Create a uniquely named test account and sign in.
- Add a workout and update at least one profile field.

- Sign out, clear browser cache/site data, and open the site in a new private window.
- Sign in again with the same account.
- Confirm the profile and workout reappear from MongoDB.
- Optionally redeploy the same release and repeat the sign-in check to confirm redeployment does not erase database data.

9. Frontend Implementation

The frontend was developed with React and Vite as a responsive Fitness Tracker interface, including reusable components for the user profile, workout entry, workout history, statistics, and the generated workout plan.

- Profile workflow collects name, username, password, age, sex, and weight; weight is used to generate a five-day workout plan.
- The workout dashboard records exercises, sets, repetitions, and weight, and calculates activity statistics; users can review history and delete individual entries.
- The frontend integrates with the backend registration endpoint (POST /user/register) using credentialed JSON requests, so the backend can create the user in MongoDB and return an httpOnly authentication cookie.
- Registration errors are surfaced clearly in the UI, and successful registration updates the active profile.

9.1 Containerization

The React application is built as a production container using a multi-stage Docker build. Nginx serves the compiled frontend and acts as the internal reverse proxy for /user, /api, and /health, so the browser never talks to the backend or MongoDB directly. Health checks, persistent MongoDB storage, generated local development secrets, JSON application logging, and restart behavior were added so the stack is repeatable across Windows, Linux, and other Docker-supported platforms.

10. Incident and Resolution Log

The following table captures the significant failures observed while implementing the final AWS architecture. Error messages are retained as part of the project's troubleshooting record; the Resolution column describes the durable correction applied.

Observed symptom	Cause	Durable resolution
Terraform provider download stalled; process later reported Killed	Large provider download under WSL with insufficient local resource headroom	Configured a persistent plugin cache, verified provider archives manually, increased EC2 root volume, reran init with portable lock-file hashes
mkdir: cannot create directory '/registry.terraform.io': Permission denied	TF_PLUGIN_CACHE_DIR was empty, so the path expanded from filesystem root	Set TF_PLUGIN_CACHE_DIR=\$HOME/.terraform.d/plugin-cache before creating the provider directory
Terraform lock file changed across Windows/Linux	Platform-specific hashes were missing	Ran initialization on Linux and committed the added Linux provider checksums
GH006: Protected branch update failed	main requires pull requests and review	Pushed to a fix branch, opened a PR, merged after approval
Database/application creation or bootstrap failure	Root disk capacity insufficient for packages, Docker images, and startup	Increased root volumes to 30 GiB and recreated affected instances from a reviewed recovery plan
Not authorized to perform sts:AssumeRoleWithWebIdentity	GitHub OIDC token subject did not match the IAM trust policy	Updated role trust conditions for the repository's main branch/production environment; reapplied Terraform
403 Forbidden pushing an ECR manifest	GitHub deployment role lacked image-upload permissions	Added the complete ECR push action set scoped to both application repositories
AccessDeniedException for ecr:DescribeImages	Deploy job verifies image tags but the role could not describe them	Added ecr:DescribeImages to the GitHub deployment role and applied the IAM plan

Website returned 502 Bad Gateway	Load balancer had no healthy application response while deployment was incomplete	Completed image publication and SSM deployment, then verified /health and /version once the target became healthy
SessionManagerPlugin is not found	AWS CLI was installed in WSL but the local Session Manager plugin was not	Installed the official Ubuntu .deb, verified session-manager-plugin, retried aws ssm start-session
Profile/workouts disappeared after browser cache clear	Frontend used browser localStorage for domain data despite workout APIs existing	Connected the UI to authenticated MongoDB-backed profile/workout endpoints; restored state from the API after login

(Some Relevant Screenshots)

```

EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
$ mkdir controllers routes config db middleware lib utils modules
● touch app.js server.js

EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
$ mkdir controllers routes config db middleware lib utils modules
● touch app.js server.js
mkdir: cannot create directory 'controllers': File exists
mkdir: cannot create directory 'routes': File exists
mkdir: cannot create directory 'config': File exists
mkdir: cannot create directory 'db': File exists
mkdir: cannot create directory 'middleware': File exists
mkdir: cannot create directory 'lib': File exists
mkdir: cannot create directory 'utils': File exists
mkdir: cannot create directory 'modules': File exists

EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
○ $

```

```
ew Go Run ... < -> group6-dockerized-3tier-app

① README.md ● JS server.js ⚙ .env

T... backend > ① README.md > abc # Fitness Tracker API (Backend Capstone ) > abc ## Option B: Run with Docker
1 # Fitness Tracker API (Backend Capstone )
242
324
336 ##### CMMYLER

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
● $ npm init -y
Wrote to C:\Users\EliteBook 745\Desktop\group6-dockerized-3tier-app\package.json:

{
  "name": "group6-dockerized-3tier-app",
  "version": "1.0.0",
  "description": "Group 6 Dockerized 3-Tier Application with full CI/CD",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/priscy-dev/group6-dockerized-3tier-app.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/priscy-dev/group6-dockerized-3tier-app/issues"
  },
  "homepage": "https://github.com/priscy-dev/group6-dockerized-3tier-app#readme"
}
```

```
EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
$
○
EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
● $ npm install express mongoose dotenv cors cookie-parser passport compression express-rate-limit

added 114 packages, and audited 115 packages in 26s

32 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
○ $ □
```

```
EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
$ mkdir controllers routes config db middleware lib utils modules
● touch app.js server.js
```

```
EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
$ mkdir controllers routes config db middleware lib utils modules
● touch app.js server.js
mkdir: cannot create directory 'controllers': File exists
mkdir: cannot create directory 'routes': File exists
mkdir: cannot create directory 'config': File exists
mkdir: cannot create directory 'db': File exists
mkdir: cannot create directory 'middleware': File exists
mkdir: cannot create directory 'lib': File exists
mkdir: cannot create directory 'utils': File exists
mkdir: cannot create directory 'modules': File exists
```

```
EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
○ $
```

```
EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
$ npm start
●
> group6-dockerized-3tier-app@1.0.0 start
> node server.js
```

```
EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
$ history | grep dev
● 44 history | grep dev

EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
○ $
```


...

HTTP

http://localhost:7000/user/register

POST

http://localhost:7000/user/register

Docs

Params

Authorization

Headers 9

Body

Scripts

Tests

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

```
1 {
2   "fullname": "Akinola",
3   "username": "akinolasamson14_del",
4   "password": "supersecret123",
5   "age": 27,
6   "sex": "Male",
7   "weight": 78
8 }
9
```

Body

Cookies 1

Headers 17

Test Results

{ } JSON

Preview

Visualize

```
1 {
2   "success": true,
3   "user": {
4     "id": "6a63964bb0c3a73129f8439d",
5     "fullname": "Akinola",
6     "username": "akinolasamson14_del",
7     "age": 27,
8     "weight": 78,
9     "sex": "Male"
10  },
11  "expiresIn": "1h"
12 }
```

GET Get data

My Collection

Getting started

GET http://localhost

GET http://localhos

GET

HTTP

http://localhost:7000/user/login

POST

http://localhost:7000/user/login

Docs

Params

Authorization

Headers 9

Body

Scripts

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

```
1 {
2   "username": "akinolasamson14_del",
3   "password": "supersecret123"
4 }
5
```

Body

Cookies 1

Headers 17

Test Results

{ } JSON

Preview

Visualize

```
1 {
2   "success": true,
3   "user": {
4     "id": "6a63964bb0c3a73129f8439d",
5     "fullname": "Akinola",
6     "username": "akinolasamson14_del",
7     "age": 27,
8     "weight": 78,
9     "sex": "Male"
10  },
11   "expiresIn": "1h"
12 }
```

0

0

```
EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
$ git pull
Already up to date.

EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
$ git tag v1.0.0
fatal: tag 'v1.0.0' already exists

EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
$ git push origin v1.0.0
Everything up-to-date

EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
$ git tag v2.0.0
git push origin v2.0.0
fatal: tag 'v2.0.0' already exists
Everything up-to-date
```

```
EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
$ git branch
git pull
git tag v1.0.0
git push origin v1.0.0 _
git push origin v2.0.0
adding-the-backend-code
* main
remote: Enumerating objects: 27, done.
remote: Counting objects: 100% (25/25), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 19 (delta 5), reused 12 (delta 2), pack-reused 0 (from 0)
Unpacking objects: 100% (19/19), 8.02 KiB | 26.00 KiB/s, done.
From https://github.com/priscy-dev/group6-dockerized-3tier-app
  115803b..9f29639  main          -> origin/main
  * [new branch]    ci-cd         -> origin/ci-cd
  * [new branch]    priscy-dev-patch-1 -> origin/priscy-dev-patch-1
Updating 115803b..9f29639
Fast-forward
 .github/workflows/ci-cd.yml | 250 ++++++
+++
 README.md                  | 26 +---
 database/README.md         | 1 -
 docker-compose.prod.yml    | 8 ++
4 files changed, 282 insertions(+), 3 deletions(-)
create mode 100644 .github/workflows/ci-cd.yml
delete mode 100644 database/README.md
create mode 100644 docker-compose.prod.yml
fatal: tag 'v1.0.0' already exists
Everything up-to-date
fatal: tag 'v2.0.0' already exists
Everything up-to-date

EliteBook 745@TEMI4515 MINGW64 ~/Desktop/group6-dockerized-3tier-app (main)
```

11. Incident Report (Brief-Required Format)

Symptom

During an early AWS release run, the deployment job failed with an authentication error, and separately a completed deployment briefly returned 502 Bad Gateway on the public endpoint, observed via the GitHub Actions run logs and a failing curl against /health.

Investigation trail

- Checked the failed job's logs — isolated the failure to the AWS OIDC role-assumption step, ruling out application code as the cause.
- Checked the IAM role's trust policy conditions against the GitHub OIDC token subject — found a mismatch for the repository/branch condition.
- After fixing trust policy, image push then failed with 403 — checked IAM permissions attached to the deployment role and found ECR push permissions were incomplete.
- After granting permissions, checked the load balancer target group health — the 502 corresponded to the window between image publish and the SSM deployment command completing.

Root cause

The GitHub deployment IAM role's OIDC trust policy and permission set were not fully scoped for the repository/branch and for ECR operations when the AWS pipeline was first stood up, and the 502 was an expected transient state while the new containers started rather than a separate defect.

Fix

Updated the OIDC trust policy conditions to match the correct repository and branch/environment, added the full required ECR permission set (including `ecr:DescribeImages`) to the deployment role, reapplied the Terraform IAM plan, and verified with a clean release: validation, build, and SSM deployment stages all completed successfully, and /health and /version returned correctly once the target became healthy.

Design reflection

The Phase 0 decision to use OIDC and SSM instead of static SSH keys made this class of failure easier to diagnose, not harder: every failure surfaced as an explicit, named AWS permission or trust error in the GitHub Actions log, rather than a silent SSH connection failure. The tradeoff is that IAM/OIDC configuration has more moving parts to get right initially. If repeating the design, the team would provision the OIDC trust policy and full ECR permission set together in the same Terraform change from the start, rather than iterating on permissions after the first failed run.

12. Operations Runbook

12.1 Verify Service Status

```
curl -i https://<application-domain>/health
curl -i https://<application-domain>/version
aws ssm describe-instance-information --region ca-central-1 --output table
```

12.2 Connect Without a Bastion

```
curl https://s3.amazonaws.com/session-manager-
downloads/plugin/latest/ubuntu_64bit/session-manager-plugin.deb -o /tmp/session-
manager-plugin.deb
sudo dpkg -i /tmp/session-manager-plugin.deb
aws ssm start-session --target <instance-id> --region ca-central-1
```

12.3 Stop, Start, and Redeploy

- Start the database instance first and wait for EC2/SSM status checks.
- Start the application instance and wait for status checks.
- Confirm Docker containers are running through SSM.
- Check /health and /version.
- If images or release configuration changed, run the approved Release or Rollback workflow rather than making untracked manual container changes.

Stopping an EC2 instance stops compute charges, but EBS volumes, the load balancer, NAT gateway, public IPv4 resources, and Route 53 can continue to incur charges.

12.4 Destroying the Environment

Destructive operation: terraform destroy removes managed infrastructure and can make the public site unavailable. Export or snapshot required database data first.

```
cd infrastructure/terraform/envs/dev
terraform plan -destroy -out=destroy.tfplan
terraform show destroy.tfplan
terraform apply destroy.tfplan
```

13. Cost Controls

A prior rough planning estimate for the complete architecture was approximately US\$23–28 for a continuously running week; the AWS Billing and Cost Management console is authoritative, and Free Tier eligibility must be checked directly in the account.

- Create an AWS Budget and billing alarm before the demonstration window.
- Review Cost Explorer by service and region (ca-central-1) each day.
- Stop EC2 instances outside the demonstration window where acceptable, while noting shared infrastructure and storage continue to cost money.
- Destroy the environment after assessment only after backing up required MongoDB data.
- Explicitly decide whether to delete or retain snapshots, detached EBS volumes, ECR images, load balancers, NAT gateways, public IPv4 addresses, and DNS records.

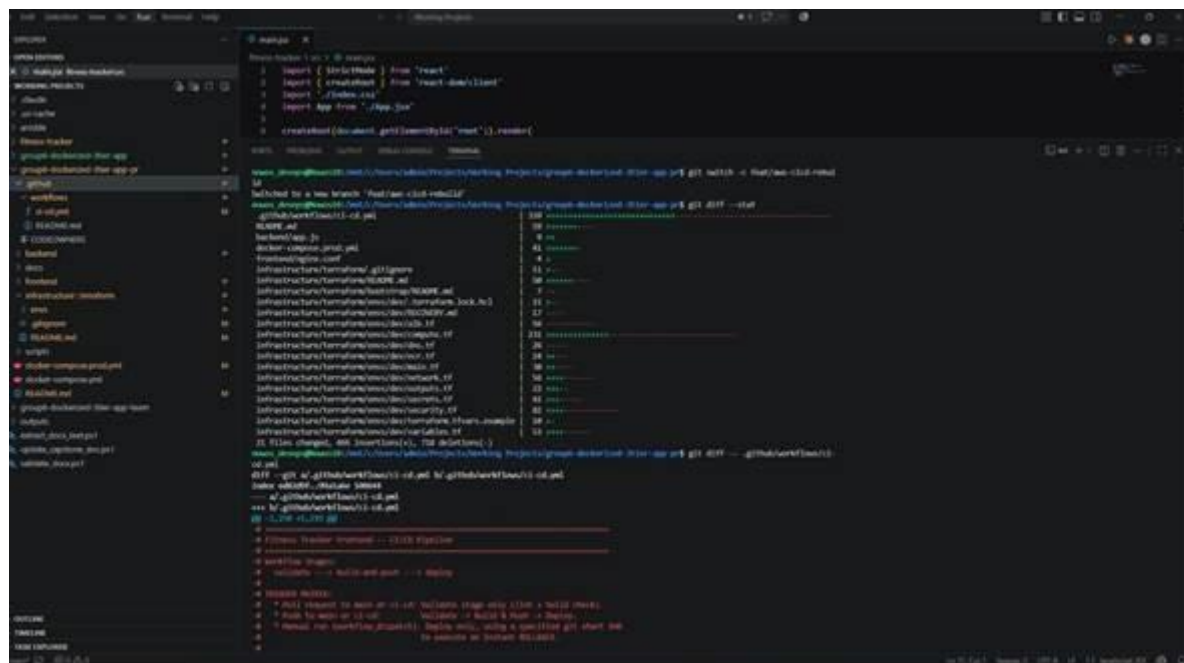
14. Final Verification Checklist

Area	Acceptance check	Status
CI	Pull-request validation passes	☒ Verified
Release	Frontend and backend images share the same immutable Git SHA	☒ Verified
OIDC	AWS credentials assumed without stored AWS access keys	☒ Verified
SSM	Application and database instances report Online	☒ Verified
Load balancer	Target is healthy and HTTPS responds	☒ Verified
Application	/health succeeds and /version matches the deployed SHA	☒ Verified
Persistence	Account, profile, and workout survive cache clearing and sign-in	☒ Verified
Rollback	A known-good prior SHA is recorded and its two ECR images exist	☒ Verified
Security	No SSH/bastion exposure and no secrets committed	☒ Verified
Cost	Budget/alarm configured and teardown responsibility assigned	☐ Confirm before submission

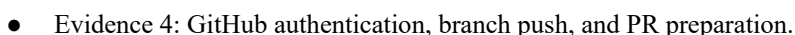
15. Evidence and Screenshots

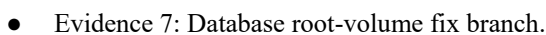
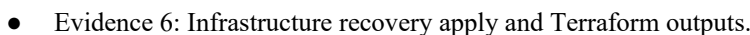
The following evidence was captured across the project's implementation and is referenced here by description; insert the corresponding screenshot images from the team's evidence folder under each heading before final submission.

- Evidence 1: Initial pipeline scope clarification — identified that the earlier pipeline did not consistently publish both backend and frontend artifacts.
- Evidence 2: AWS CI/CD rebuild branch and change scope — dedicated feat/aws-cicd-rebuild branch capturing workflow, Docker Compose, Terraform, and documentation changes.



- Evidence 3: Workflow replacement from SSH/Docker Hub to AWS deployment — diff showing the transition to the three-stage AWS pipeline.



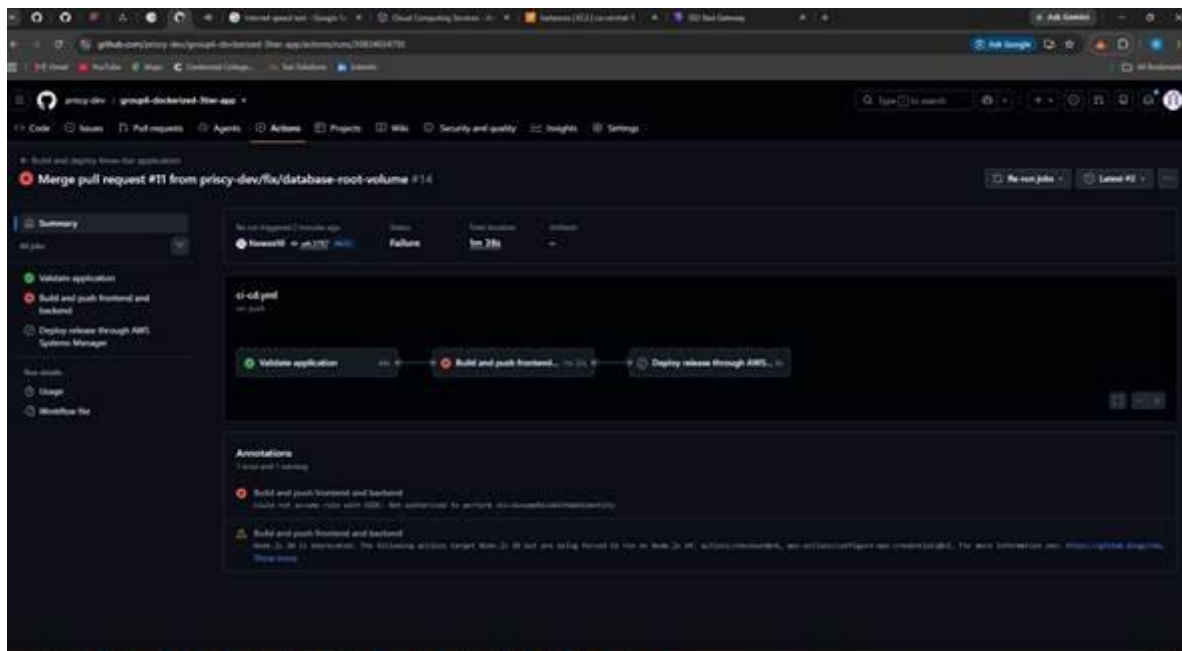


```

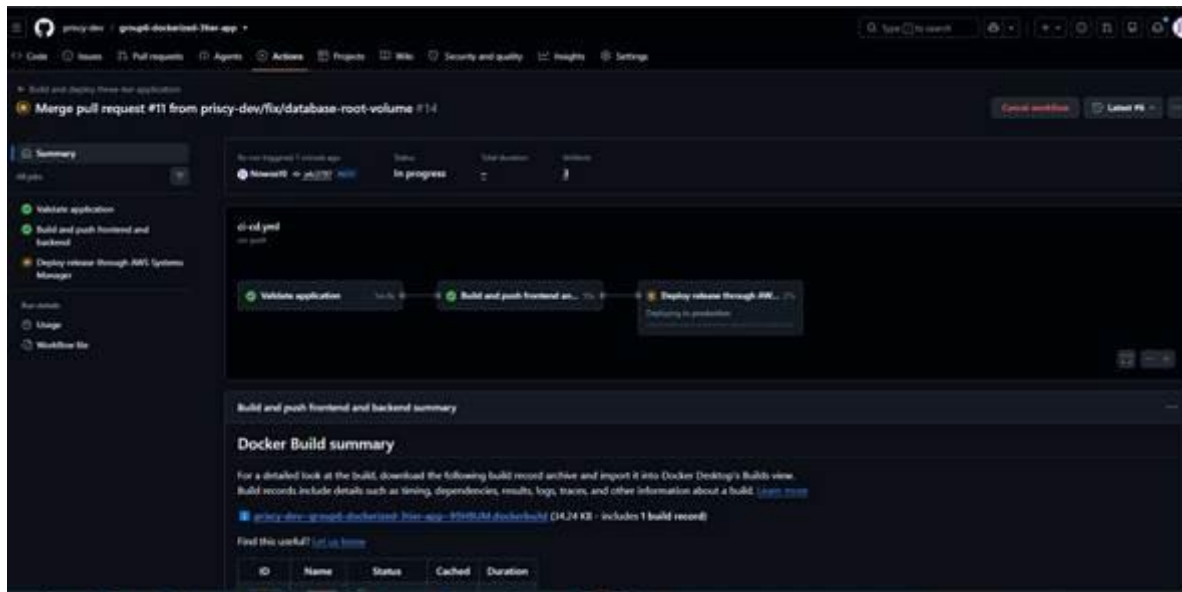
git push -u origin fix/database-root-volume
Counting objects: 11, done.
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 566 bytes | 42.00 KiB/s, done.
Total 7 (delta 4), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
remote: Create a pull request for 'fix/database-root-volume' on GitHub by visiting:
remote:   https://github.com/prisy-dev/group-dockerized-Blir-app/pull/new/fix/database-root-volume
remote: To https://github.com/prisy-dev/group-dockerized-Blir-app.git
 * [new branch]   fix/database-root-volume -> fix/database-root-volume
branch 'fix/database-root-volume' set up to track 'origin/fix/database-root-volume'.
main, devops@aws31b:mt/~/Users/admin/Projects/working_projects/group-dockerized-Blir-app-pr/infrastructure/terraform/main$ git push -u origin fix/database-root-volume

```

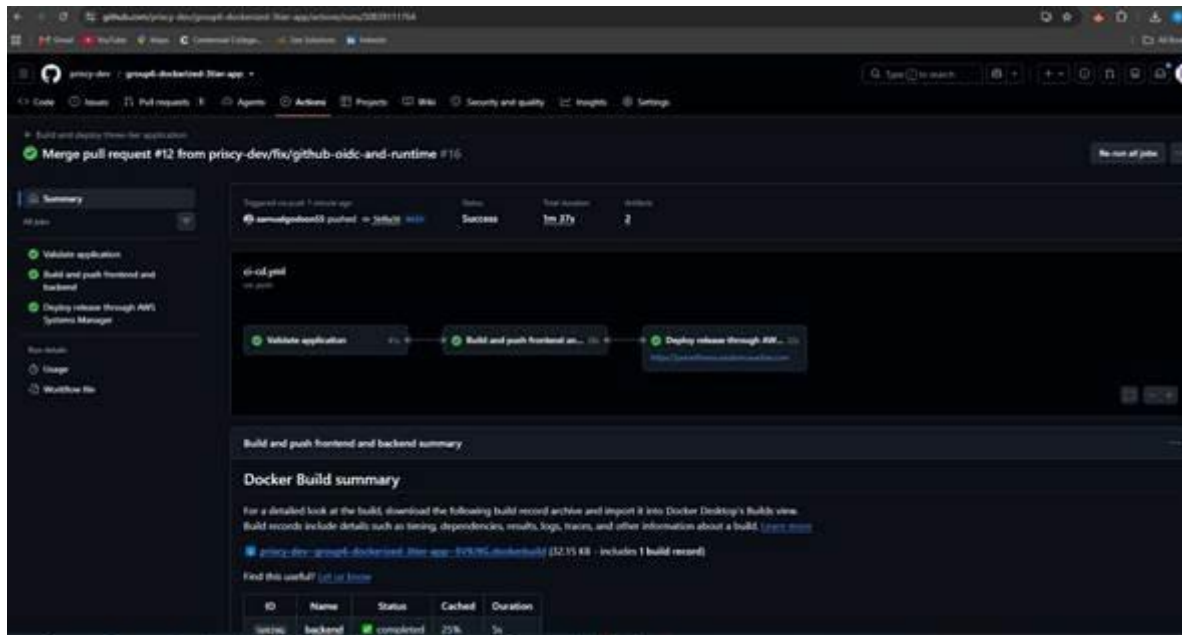
- Evidence 8: GitHub OIDC trust failure — isolated to the IAM trust relationship rather than application code.



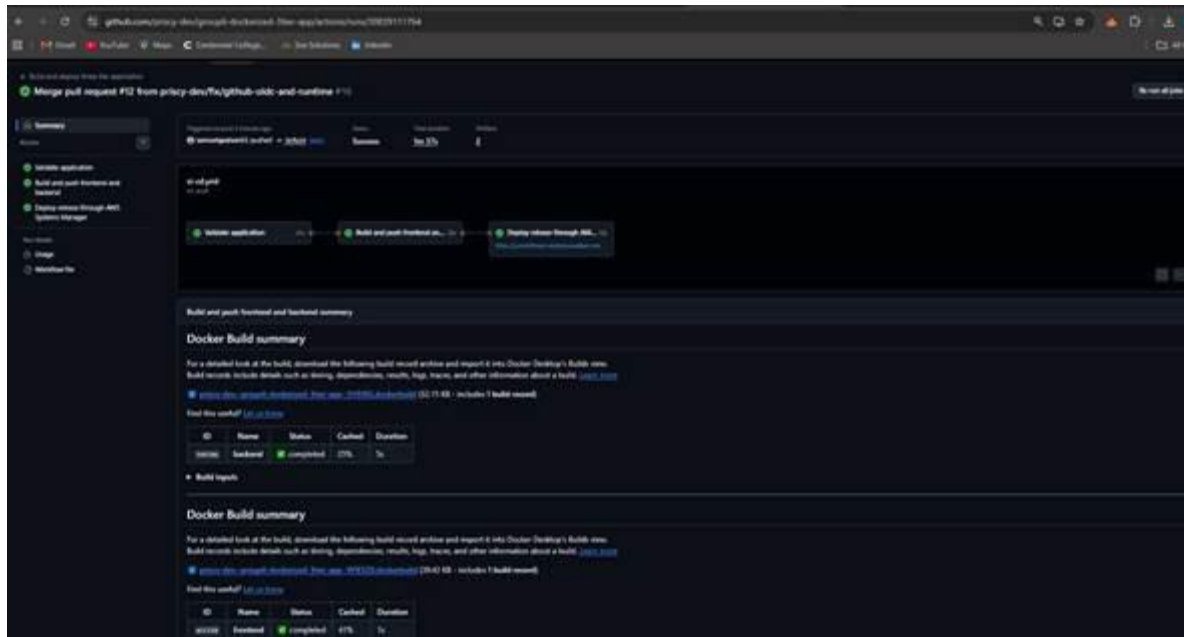
- Evidence 9: Successful validation and image build; deployment in progress.



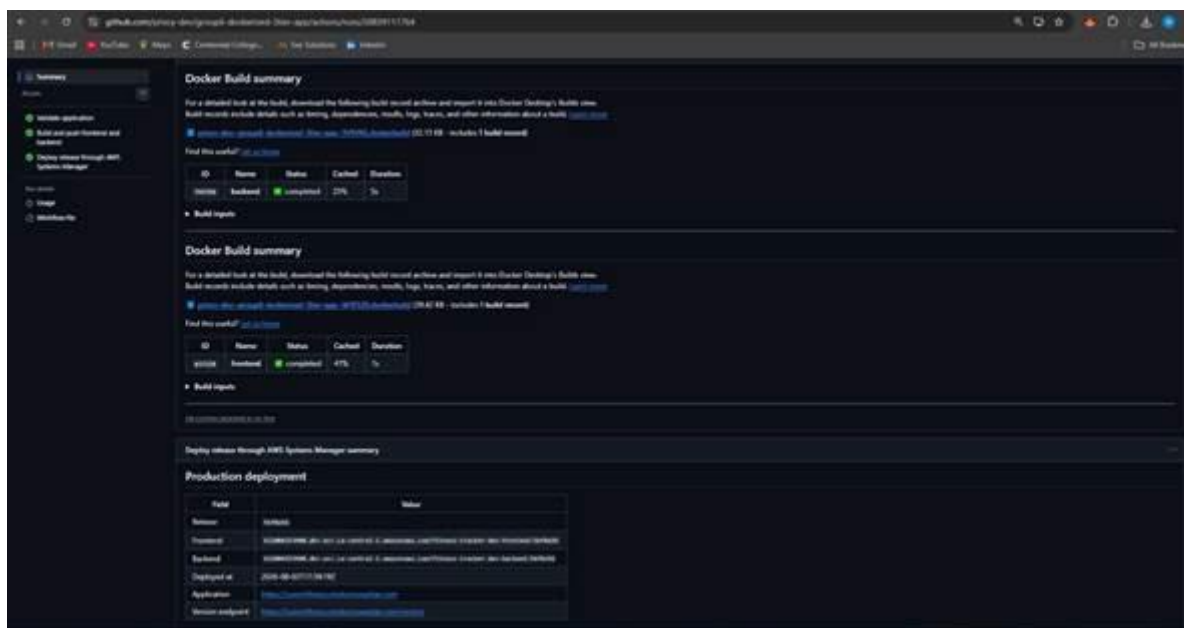
- Evidence 10: Completed release workflow — all three release jobs green.



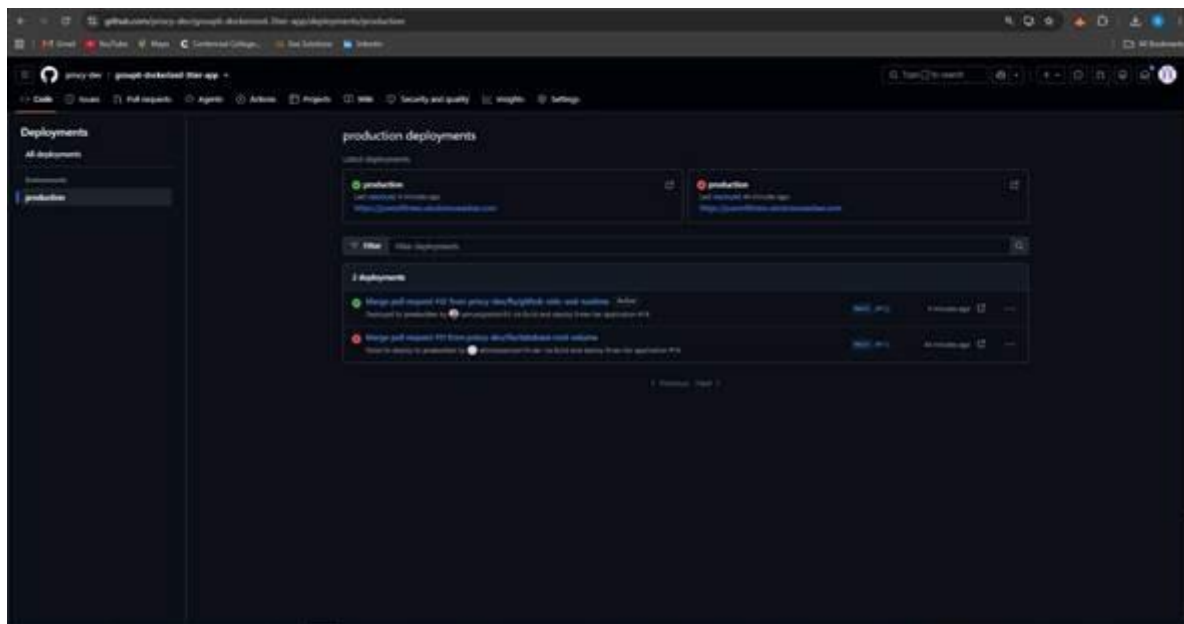
- Evidence 11: Successful frontend and backend image builds.



- Evidence 12: Production deployment summary and release identity (release 5bf8a56).



- Evidence 13: GitHub production deployment history.



Production Verification (Live Endpoints)

- Screenshot 1: /health returning {"status":"ok"}

Pretty-print ☐

```
{"version":"769868b497b3bb2844ede8a1487e684059b89700","frontendImage":"622004253908.dkr.ecr.ca-central-1.amazonaws.com/fitness-tracker-dev-frontend:769868b497b3bb2844ede8a1487e684059b89700","backendImage":"622004253908.dkr.ecr.ca-central-1.amazonaws.com/fitness-tracker-dev-backend:769868b497b3bb2844ede8a1487e684059b89700","deployedAt":"2026-08-03T19:09:15Z"}
```

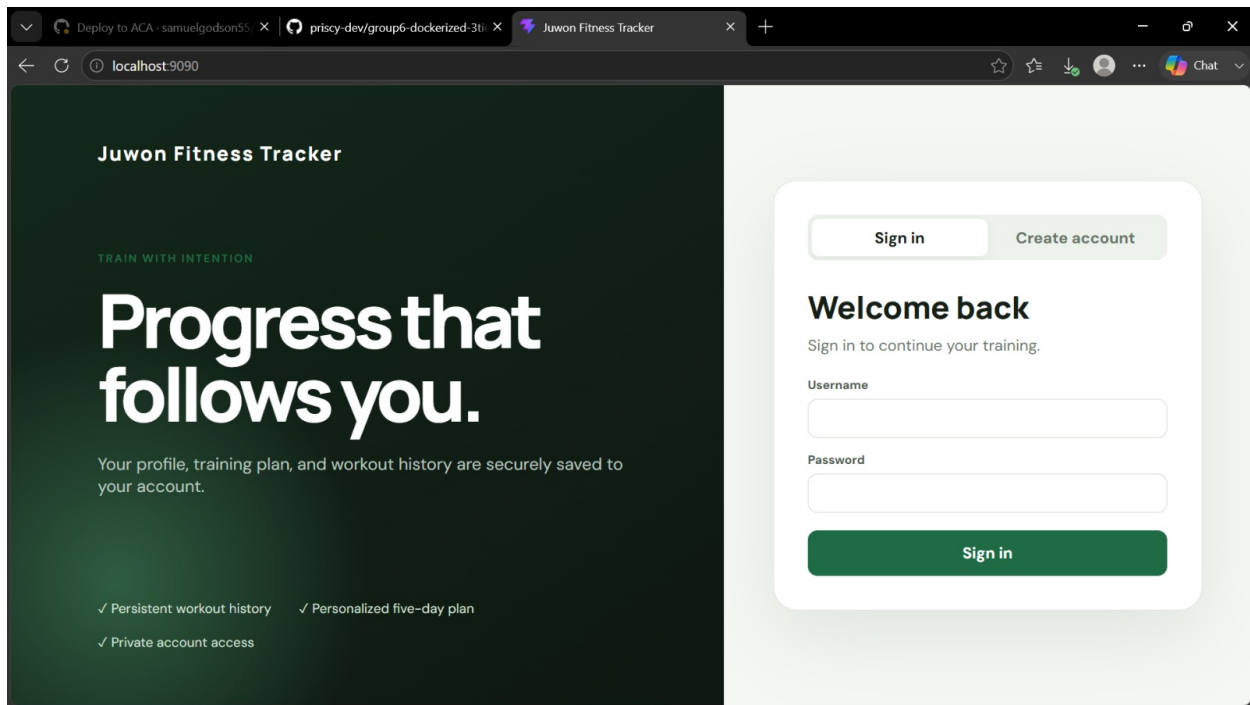
Pretty-print ☐

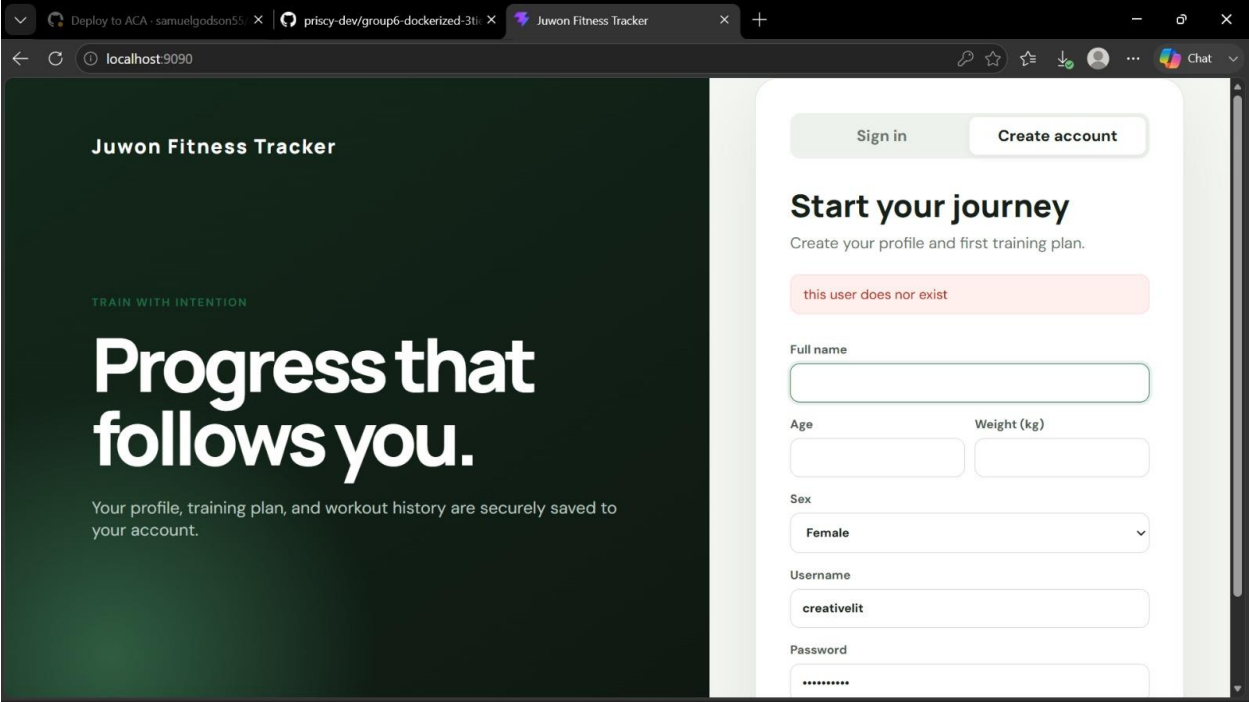
```
{"status":"ok"}
```


- Local frontend UI and Local Docker Compose starting

```
Windows PowerShell
#24 DONE 0.1s

#25 [frontend] resolving provenance for metadata file
#25 DONE 0.1s
[+] up 7/7
✔Image group6-dockerized-3tier-app-main-backend      Built      10.8s
✔Image group6-dockerized-3tier-app-main-frontend      Built      10.8s
✔Network group6-dockerized-3tier-app-main_default     Created     0.4s
✔Volume group6-dockerized-3tier-app-main_mongo_data   Created     0.1s
✔Container fitness_mongo                             Created     0.8s
✔Container fitness_backend                            Created     0.4s
✔Container fitness_frontend                           Created     0.4s
Attaching to fitness_backend, fitness_frontend, fitness_mongo
Container fitness_mongo Waiting
fitness_mongo | about to fork child process, waiting until server is ready for connections.
fitness_mongo | forked process: 30
fitness_mongo |
fitness_mongo | {"t":{"$date":"2026-08-05T13:01:22.939+00:00"},"s":"I", "c":"CONTROL", "id":20698, "ctx":"main","msg":"***** SER
VER RESTARTED *****"}
fitness_mongo | {"t":{"$date":"2026-08-05T13:01:23.020+00:00"},"s":"I", "c":"NETWORK", "id":4915701, "ctx":"main","msg":"Initializ
ed wire specification", "attr":{"spec":{"incomingExternalClient":{"minWireVersion":0,"maxWireVersion":21},"incomingInternalClient":{"m
inWireVersion":0,"maxWireVersion":21},"outgoing":{"minWireVersion":6,"maxWireVersion":21},"isInternalClient":true}}}
fitness_mongo | {"t":{"$date":"2026-08-05T13:01:23.023+00:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"main","msg":"Starting
thread pool", "attr":{"poolName":"parallel execution pool", "numThreads":0, "minThreads":0, "maxThreads":128}}
fitness_mongo | {"t":{"$date":"2026-08-05T13:01:23.026+00:00"},"s":"I", "c":"CONTROL", "id":23285, "ctx":"main","msg":"Automatic
ally disabling TLS 1.0, to force-enable TLS 1.0 specify --sslDisabledProtocols 'none'"}
fitness_mongo | {"t":{"$date":"2026-08-05T13:01:23.056+00:00"},"s":"I", "c":"NETWORK", "id":4648601, "ctx":"main","msg":"Implicit
TCP FastOpen unavailable. If TCP FastOpen is required, set tcpFastOpenServer, tcpFastOpenClient, and tcpFastOpenQueueSize."}
fitness_mongo | {"t":{"$date":"2026-08-05T13:01:23.114+00:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"main","msg":"Starting
thread pool", "attr":{"poolName":"OCSPCache", "numThreads":0, "minThreads":0, "maxThreads":8}}
fitness_mongo | {"t":{"$date":"2026-08-05T13:01:23.114+00:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"main","msg":"Starting
thread pool", "attr":{"poolName":"ThreadPool2", "numThreads":0, "minThreads":0, "maxThreads":1}}
fitness_mongo | {"t":{"$date":"2026-08-05T13:01:23.115+00:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"main","msg":"Starting
thread pool", "attr":{"poolName":"ThreadPool1", "numThreads":0, "minThreads":0, "maxThreads":1}}
fitness_mongo | {"t":{"$date":"2026-08-05T13:01:23.115+00:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"main","msg":"Starting
```





Additional screenshots to attach per the brief's submission checklist: successful local docker compose up, successful and blocked connectivity/API tests in Postman, a demonstrated and screenshotted rollback, and the live application accessible over HTTPS.

16. Software and Tools Used

Tool	Role
Git Bash	Terminal for commands, navigation, and scripts on Windows
Node.js & npm	JavaScript runtime and package manager for backend and frontend
MongoDB (via Mongoose)	NoSQL database for users, workouts, and auth data
Postman	Independent API testing of backend endpoints
Vite	Frontend build tool and dev server for the React UI
Docker / Docker Compose	Containerization and local multi-service orchestration
Terraform	Infrastructure as code for the AWS environment
AWS (EC2, ECR, ALB, Route 53, ACM, Secrets Manager, SSM, S3)	Production hosting, registry, HTTPS, secrets, and deployment
GitHub / GitHub Actions	Version control, branch protection, CI/CD automation

17. Conclusion

Group 6 delivered a fully containerized, three-tier Fitness Tracker application with an automated, reviewed CI/CD pipeline and a production-grade AWS deployment. The team's design evolved deliberately from a simple Docker Hub/SSH baseline to a Terraform-managed, OIDC-authenticated, SSM-deployed architecture, with each stage tested, broken, diagnosed, and fixed rather than assumed to work. The final system satisfies the brief's core requirements: traceable versioning via immutable Git-SHA image tags, a demonstrated rollback path, no hardcoded or committed secrets, public access restricted to the required port, and a documented incident with root cause, fix, and design reflection.